

Online Self-Calibrating Steering Control Using Recursive Least Squares with IMU Feedback on a Small-Scale Autonomous Vehicle

Pham Xuan Tung ¹[0009-0005-9811-8027] and Hoang Thai Duong ¹ [0009-0006-1872-9133]

¹ University of Science and Technology of Hanoi, Vietnam Academy of Science and Technology, Hanoi 100000, Vietnam
pham-xuan.tung@usth.edu.vn

Abstract. This paper presents an online self-calibrating steering control for small-scale autonomous vehicles to continuously correct mechanical deviations and surface changes. Applying a Recursive Least Squares (RLS) algorithm with exponential forgetting and real-time integrated IMU feedback data, the vehicle automatically cancels systematic steering errors. **Experimental results show that RLS does not outperform PID or MPC under static straight-line conditions. Its main benefit appears under perturbed operating conditions, where it achieves the lowest mean yaw error among the tested controllers. The paper also evaluates a lightweight error-pattern diagnosis heuristic using synthetic telemetry with known-by-construction labels; validation against independently confirmed hardware fault cases remains future work.**

Keywords: recursive least squares, online calibration, steering control, IMU feedback, autonomous vehicles, adaptive control

1 Introduction

In autonomous vehicle applications, precise lateral tracking relies heavily on the reliability of the steering system calibration model [1-3]. On small-scale robotic platforms, which is widely used for algorithmic experimentation, maintaining absolute mechanical symmetry is a major challenge. Elements such as play in rack and pinion joints, uneven friction between wheels, manufacturing tolerances, and battery voltage drop over time create nonlinear, asymmetrical steering errors. These uncalibrated system characteristics lead to heading drift even when the steering control command is set to neutral (zero), severely degrading lane-keeping and tracking performance.

The traditional solution for addressing steering errors is to perform batch offline calibration, for example, running open-loop sweep scenarios to build a static least squares (OLS) function between the target yaw angular velocity and the input control command [4, 5]. While effective immediately after calibration, this method reveals significant weaknesses. First, there is time-varying degradation; the system cannot respond to dynamic changes occurring while the vehicle is in motion, such as changes in load, changes in road surface friction coefficient, or mechanical wear of the actuators. Secondly, there is operational inefficiency; the system requires frequent vehicle stops to

rerun the experimental sweep scenarios on a simulator, compromising the vehicle's long-term autonomy. On the other hand, the rigid model causes the regression parameters to freeze and become fixed, treating the vehicle's dynamic system as invariant, leading to the accumulation of directional integral errors over long distances.

To address the real-time adaptation problem within the constraints of embedded hardware resources, this paper offers three core contributions. First is an online auto-calibration architecture framework, which facilitates the design and implementation of an active parameter identification loop using RLS with obsolescence [6-9]. **This architecture completely eliminates reliance on static lookup models by continuously updating the system matrix within the high-frequency control loop.** Next is an algorithm for classifying and diagnosing end-segment error patterns, which helps construct a set of mathematical segmentation indices based on error growth rate and linear slope to classify telemetry patterns consistent with near-fixed bias, growing late-run drift and mixed behavior [10]. Finally, the system was evaluated on real hardware using four controller configurations: offline linear regression (LR), classical PID, adaptive online RLS, and lightweight model predictive control (MPC), under static and perturbed operating conditions.[11,12].

2 System description

2.1 Overview of AutoCar III Hardware

The robotic validation vehicle used in this study is the Hanback Electronics AIoT AutoCar III, an integrated, small-scale autonomous driving platform configured to perform real-time edge computing. The vehicle is built around a dual-processor distributed network architecture consisting of an upper-level Application Processor (AP) and a lower-level Microcontroller Unit (MCU).

Embedded systems convert raw register data into standard SI units. The accelerometer measures linear acceleration along three axes, while the gyroscope measures rotational velocity around the vehicle axes. The sensor's fusion algorithm combines gyroscope, accelerometer, and digital compass data to estimate Euler angles, particularly the yaw angle for determining heading deviation. The vehicle is powered by an integrated Lithium-ion battery rated at 14.8 V nominal and 16.8 V maximum; its voltage is continuously monitored by the MCU through an internal ADC channel using data extracted from CAN frames.

2.2 Existing Calibration Approach (Baseline)

The manufacturer's default steering calibration uses an offline linear regression baseline, assuming that vehicle dynamics remain constant. Before operation, the vehicle travels at a constant speed while the steering command is sequentially varied across predefined angles. After each command, the system waits 0.5 s for stabilization, then records the yaw rate from the IMU and the corresponding steering input. An Ordinary Least Squares (OLS) model is fitted to these data to obtain a fixed steering parameter

vector. Because the parameters are not updated during operation, the model cannot adapt to changing vehicle dynamics or compensate for dynamic errors.

3 Proposed Approach

3.1 Problem Formulation

The core objective of the proposed control structure is to establish a highly accurate horizontal tracking mechanism for the model autonomous vehicle in straight-line road travel scenarios, even when serious geometric asymmetry deviations of the mechanical system exist. Let u_k denote the normalized steering-angle command transmitted from the edge processor to the actuator at time step k . The relationship between the steering system control command and the actual horizontal dynamic feedback of the chassis is represented by a parameterized linear model:

$$u_k = \theta_k^T x_k + \delta_{bias} \quad (1)$$

In this case, x_k is the multi-sensor state characteristic vector extracted from the IMU, θ_k is the system identification weight matrix, and δ_{bias} represents the mechanical differential static error components. For a steady-state operation scenario, the ideal dynamic conditions require that the target's angular velocity around the vertical axis (yaw rate) is completely zero ($\psi_{ref}=0$) and the absolute direction of movement remains unchanged ($\psi_{ref}=\psi_0=0$). Any angular velocity $\dot{\psi}_z \neq 0$ or directional error $\psi_{error} \neq 0$ appearing in the telemetry stream are considered non-stationary structural disturbances. Therefore, the online calibration problem is reduced to continuously optimizing and updating the parameter vector θ_k in real time to generate a dynamic steering compensation, forcing inertial state errors towards absolute equilibrium.

3.2 RLS with Forgetting Factor

To adapt flexibly to dynamic variations occurring over time (such as battery voltage drop or thermal structural drift), the algorithm applies a progressively decreasing penalty function to historical data. The smoothing least squares optimization criterion at time step k is defined as follows:

$$J_k(\theta) = \sum_{i=1}^k \lambda^{k-i} (y_i^* - x_i^T \theta)^2 \quad (2)$$

where $\lambda \in (0, 1]$ is the exponential forgetting factor that controls the trade-off between the speed of tracking disturbances and the stability of the identification parameters. The real-time recursive update loop completely eliminates the inverse operations of dense matrices, which are executed sequentially at each control cycle.

The priori prediction is calculated by:

$$\hat{y}_k = x_k^T \theta_{k-1} \quad (3)$$

Determine the adaptive monitoring target signal based on the error-driven monitoring configuration, the modified signal is synthesized directly by correcting the prior values through error feedback from the gyro and euler:

$$y_k^* = \hat{y}_k - \gamma_{\text{gyro}} \cdot \dot{\psi}_z - \gamma_{\text{euler}} \cdot \psi_{\text{error}} \quad (4)$$

where γ_{gyro} and γ_{euler} are adaptive error gain coefficients aimed at guiding the model towards error elimination.

The innovation error (ξ_k) and adaptive Kalman gain vector (K_k) are calculated as:

$$\xi_k = y_k^* - \hat{y}_k \quad (5)$$

$$K_k = \frac{P_{k-1} x_k}{\lambda + x_k^T P_{k-1} x_k} \quad (6)$$

Update the steering parameter weight vector online:

$$\theta_k = \theta_{k-1} + K_k \xi_k \quad (7)$$

Update the recursive error covariant matrix (P_k):

$$P_k = \frac{1}{\lambda} (P_{k-1} - K_k x_k^T P_{k-1}) \quad (8)$$

Before operation, the system can initialize the parameter vector θ_0 using the offline LR estimate θ_{static} . The initial covariance is set to $P_0 = \delta_0 I$, accelerating convergence and improving early-stage stability.

3.3 Multi-Sensor Feature Vector

The system architecture separates the input feature vector structure x_k into three independent modes based on the sensor trigger flag configuration, allowing for the evaluation of the tracking performance of each isolated sensor hardware array. A fixed bias constant component with a value of 1.0 is always inserted at the end of the vector to completely isolate fixed mechanical geometric deviations from state kinematic variations.

The adaptive identification core incorporates protection function. This function continuously monitors the number of active sensor flags at runtime. If the input sensor configuration is abruptly changed by the operator, the system will automatically restructure the covariance matrix size P_k and the corresponding weight vector θ_k without crashing the control execution flow.

3.4 Online Calibration Loop

The online steering-calibration loop is executed on the edge processor at a sampling interval of $\Delta t = 0.15$ s. At each iteration, four functional modules are sequentially evaluated to ensure smooth and safe actuator operation.

Continuous Euler-Angle Unwrapping

To eliminate discontinuities when the yaw angle crosses the $0^\circ/360^\circ$ boundary, the raw Euler angle is continuously unwrapped. The angular offset O_k is updated as

$$\Delta\psi_{\text{raw},k} = \psi_{\text{raw},k} - \psi_{\text{raw},k-1} \quad (9)$$

$$O_k = \begin{cases} O_{k-1} - 360^\circ, & \Delta\psi_{\text{raw},k} > 180^\circ \\ O_{k-1} + 360^\circ, & \Delta\psi_{\text{raw},k} < -180^\circ \\ O_{k-1}, & \text{otherwise} \end{cases} \quad (10)$$

resulting in the continuous yaw angle

$$\psi_{unwrapped,k} = \psi_{raw,k} + O_k \quad (11)$$

Complementary Heading-Fusion Filter

A complementary filter combines the unwrapped Euler heading with the integrated gyroscope signal. This approach suppresses high-frequency noise in the Euler-angle measurement while limiting long-term gyroscope drift:

$$\psi_{fused,k} = \alpha(\psi_{fused,k-1} + \dot{\psi}_{z,k}\Delta t) + (1 - \alpha)\psi_{unwrapped,k} \quad (12)$$

where α denotes the filter weighting coefficient.

Adaptive Steering Slew-Rate Limiter

A dynamic slew-rate limiter is applied to prevent abrupt steering reversals caused by sensor disturbances. The allowable steering increment is increased during high-rate maneuvers according to

$$\Delta u_{max,k} = \left[R_{base} \max \left(1, \min \left(2, \max \left(\frac{\psi_{error,k}}{\psi_{thr}}, \frac{\dot{\psi}_{z,k}}{\dot{\psi}_{thr}} \right) \right) \right) \right] \Delta t \quad (13)$$

where R_{base} is the nominal steering slew rate, ψ_{thr} and $\dot{\psi}_{thr}$ are the corresponding activation thresholds. The requested steering command is then constrained as

$$u_{slew,k} = u_{final,k-1} + \text{clip}(u_{requested,k} - u_{final,k-1}, -\Delta u_{max,k}, \Delta u_{max,k}) \quad (14)$$

Command Synthesis and Mechanical Limiting

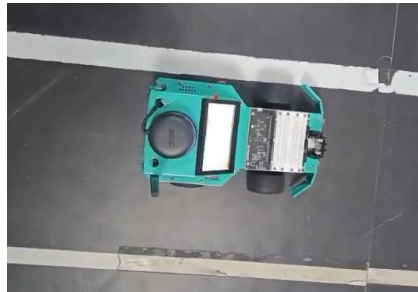
The final steering command combines the slew-limited output with the RLS-estimated steering gain, adaptive linear-deviation compensation, and linear stability-hold term:

$$u_{final,k} = \text{clip}(u_{slew,k}G_{steer} + u_{trim} + u_{hold}, U_{min}, U_{max}) \quad (15)$$

Here, U_{min} , U_{max} define the actuator's mechanical limits, preventing command saturation and steering-geometry jamming in the AutoCar III platform.

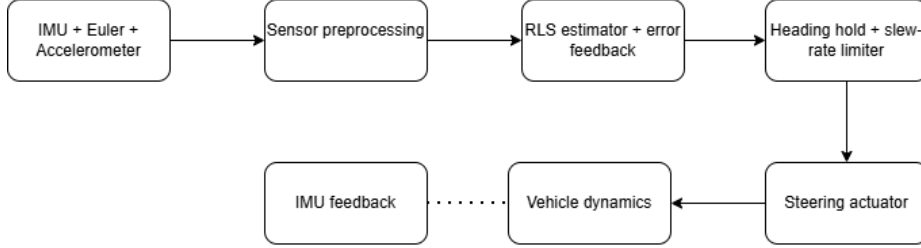
4 Experimental Setup

Fig. 1. Electronics AloT AutoCar III



Tracking experiments were conducted on a level indoor track. At the beginning of each trial, the vehicle was positioned at the track origin with its chassis aligned parallel to the centerline. The telemetry system was then initialized, followed by a stationary bias calibration using the first 40 sensor samples. The vehicle subsequently traveled at a prescribed speed and direction for a predefined duration. At the endpoint, the lateral path deviation was measured manually, with optional segment offsets entered through the diagnostic terminal.

Fig. 2. System architecture



Unless otherwise stated, the controller used a sampling interval of 0.15 s, an initial RLS covariance scale of 100, a steering gain of 0.9, an RLS error-feedback gain of 0.01, and steering limits of $[-1, 1]$. The complementary heading filter used $\alpha = 0.92$. The default slew-rate limits were 1.2 per second under nominal error and 3.0 per second under high error, with activation thresholds of 6° heading error and 0.8 deg/s yaw rate. The heading-hold gains were 0.010 for heading error and 0.003 for yaw rate.

4.1 Baseline Methods

The proposed method was compared with three baseline controllers: an offline linear regression model with fixed parameters, a classical PID controller based on instantaneous yaw-rate error, and a lightweight model predictive controller using a discrete bicycle model over a finite prediction horizon.

4.2 Evaluation Metrics

Performance was evaluated using the mean absolute error (MAE) and root mean square error (RMSE) of the yaw response. Error growth ratio and slope were derived from segmented tracking histories to characterize temporal drift. Endpoint deviation and per-cycle processing latency were additionally measured to assess physical tracking accuracy and real-time computational efficiency.

5 Result and Discussion

5.1 Straight-Line Calibration

Table 1. Straight-line calibration results

Method	MAE (deg)	RMSE (deg)	Compute Time (ms/step)
LR	0.043 ± 0.020	0.056	0.988
PID	0.023 ± 0.010	0.029	1.228
RLS	0.046 ± 0.017	0.059	2.499
MPC	0.023 ± 0.006	0.027	8.652

Under static linear movement conditions, PID and MPC achieve the lowest yaw errors (Table 1). MPC provides the best tracking accuracy, but at the cost of significantly higher computational overhead. PID offers a strong balance between performance and efficiency. RLS performs slightly worse in this static scenario, indicating that its core benefit lies not necessarily in better steady-state accuracy, but in its ability to adapt online as environmental conditions change.

Table 2. RLS performance for different forgetting factors

λ	MAE (deg)	RMSE (deg)	Convergence Time (s)
0.90	0.020 ± 0.004	0.028	0.78
0.92	0.019 ± 0.004	0.028	0.69
0.95	0.026 ± 0.005	0.031	0.74

Among the tested values, $\lambda = 0.92$ produced the lowest observed MAE and RMSE. However, its difference from $\lambda = 0.90$ is small relative to the reported run-to-run variability; therefore, $\lambda = 0.92$ is selected as the best-performing setting in this experiment, not as a universally optimal value.

Table 3. Perturbation test results

Method	MAE (deg)	Peak (deg)	Recover Time (s)
LR	0.048 ± 0.033	0.13	0.78
PID	0.033 ± 0.004	0.07	0.78
RLS	0.026 ± 0.003	0.08	0.78

Method	MAE (deg)	Peak (deg)	Recover Time (s)
MPC	0.146 ± 0.013	0.33	0.79

RLS achieves the best yaw error under perturbation, outperforming both LR and PID (Table 3). This result demonstrates the value of online adaptation when the vehicle dynamics change unexpectedly. Although the reported recovery times are similar across methods in the summary logs, the lower MAE and lower peak yaw error of RLS indicate better robustness after disturbance. The MPC implementation performs poorly in this experiment, likely because the simplified model is not updated sufficiently to handle the changed dynamics.

Table 4. Multi-sensor comparison

Configuration	Feature Dimension	MAE (deg)	RMSE (deg)	Convergence Time (s)
Gyro-only	2	0.023 ± 0.003	0.027	0.63
Euler-only	2	0.000 ± 0.000	0.000	2.39
Full-IMU	4	0.257 ± 0.011	0.268	2.66

Table 4 show the result of multi-sensor comparison. The gyro-only configuration gives low error and fast convergence. The Euler-only configuration reports zero yaw errors in the logged summary, but this result should be interpreted cautiously because it may reflect the way the metric was computed or the behavior of the orientation signal in this specific implementation. The full-IMU configuration performs worse than expected, suggesting that simply adding more sensor channels does not guarantee better performance. In practice, multi-sensor fusion requires proper normalization, preprocessing, and feature selection. This is an important finding because it shows that the quality of feature design can matter more than the raw number of sensors.

Table 5. Validation of the temporal error-pattern classifier

Component	Description / Result
Algorithm	Three-segment analysis of $ \dot{\psi}_z $
Classes	Fixed bias; growing drift; mixed/inconclusive
Ground truth	Synthetic signals with known-by-construction temporal patterns
Sample size	180 runs; 60 per class; 5 s/run; 0.15s sampling
Metrics	Overall accuracy, per-class precision, recall, F1-score, and confusion matrix.

Component	Description / Result
Fixed-bias class	Precision = 0.950; Recall = 0.950; F1 = 0.950; Support = 60.
Growing-drift class	Precision = 0.982; Recall = 0.917; F1 = 0.948; Support = 60.
Mixed/inconclusive class	Precision = 0.875; Recall = 0.933; F1 = 0.903; Support = 60.
Overall result	Accuracy = 0.933; macro precision = 0.936; macro recall = 0.933; macro F1 = 0.934.
Confusion matrix	Rows = ground truth, columns = prediction: fixed bias [57, 0, 3]; growing drift [0, 55, 5]; mixed [3, 1, 56].

Table 5 summarizes the validation of the temporal error-pattern classifier. Each run was divided into three equal segments, and the classifier used the last-to-first growth ratio and the linear slope of the absolute yaw-rate signal to distinguish near-fixed bias, growing late-stage drift, and mixed or inconclusive behavior. The validation used 180 synthetic runs with known-by-construction labels, comprising 60 runs per class. The classifier achieved 0.933 accuracy and a macro-F1 score of 0.934. The mixed class obtained the lowest F1-score because its synthetic trend was intentionally generated near the decision boundary between fixed bias and growing drift. These results validate the software decision rule under controlled synthetic conditions, but they do not establish end-to-end fault-diagnosis accuracy on the physical vehicle because independently confirmed hardware root causes were not available.

6 Conclusion

This paper presented an online adaptive steering-calibration framework based on Recursive Least Squares for a small autonomous vehicle. Under static straight-line conditions, RLS did not achieve the lowest yaw error; PID and MPC performed better in this setting. Under perturbation, however, RLS achieved the lowest MAE among the tested controllers, supporting the use of online adaptation when vehicle dynamics change. The proposed time-segment diagnosis module was evaluated using synthetic telemetry with known-by-construction labels and achieved 0.933 accuracy and 0.934 macro-F1. These results quantify the behavior of the classification rule but do not constitute end-to-end hardware fault-diagnosis validation. Future work will validate the classifier using controlled hardware cases with independently confirmed causes, such as a known mechanical linkage offset or a controlled battery-voltage degradation profile. Additional work will automate endpoint deviation measurement using optical-flow or depth sensors and investigate nonlinear compensation based on verified fault labels

References

1. Rajamani R (2012) *Vehicle Dynamics and Control*, 2nd edn. Springer, New York.
2. Pacejka HB (2005) *Tire and Vehicle Dynamics*, 2nd edn. Elsevier, Oxford.
3. Wong JY (2008) *Theory of Ground Vehicles*, 4th edn. Wiley, Hoboken
4. Ljung L (1999) *System Identification: Theory for the User*, 2nd edn. Prentice Hall, Upper Saddle River.
5. Montgomery DC, Peck EA, Vining GG (2012) *Introduction to Linear Regression Analysis*, 5th edn. Wiley, Hoboken.
6. Haykin S (2002) *Adaptive Filter Theory*, 4th edn. Prentice Hall, Upper Saddle River.
7. Ljung L, Söderström T (1983) *Theory and Practice of Recursive Identification*. MIT Press, Cambridge.
8. Ding F, Chen T (2005) Combined parameter and output estimation of dual-rate systems using an auxiliary model. *Automatica* 41(10):1739–1748.
9. Verhaegen M, Verdult V (2007) *Filtering and System Identification: A Least Squares Approach*. Cambridge University Press, Cambridge.
10. Isermann R (2006) *Fault-Diagnosis Systems: An Introduction from Fault Detection to Fault Tolerance*. Springer, Berlin Heidelberg.
11. Åström KJ, Murray RM (2008) *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, Princeton.
12. Åström KJ, Hägglund T (2006) *Advanced PID Control*. ISA, Research Triangle Park.